

Vending Machine

Projeto da UC de Sistemas Embebidos

Grupo 8

Francisca Domingues - A104953

Joana Carvalho - A104865





01

TEMA



Tema

As máquinas de venda automática são dispositivos capazes de dispensar produtos ao utilizador mediante um pagamento prévio, sem necessidade de intervenção humana.

São amplamente utilizadas em contextos como escolas, hospitais, escritórios e espaços públicos, de modo a apresentar conveniência e disponibilidade contínua.

Objetivo

Este projeto tem como objetivo a simulação de uma máquina de venda automática, com foco no desenvolvimento da lógica de funcionamento e no controlo sequencial das operações associadas ao processo de venda.

O trabalho não contemplará a construção física do sistema, irá incidir exclusivamente na sua modelação em ambiente de simulação.



Etapas do Processo

01

Seleção e Pagamento

O utilizador escolhe o produto desejado e insere o pagamento. A máquina verifica se o valor é suficiente.

02

Processamento Interno

O microcontrolador executa o processo de controlo.

03

Dispensar Produto

A máquina usa os mecanismos de espiral/mola rotativa seguida de queda livre (o produto cai para um compartimento de recolha).

04

Troco e confirmação

Se o pagamento exceder o valor, a máquina calcula e devolve o troco automaticamente.

05

Gestão de stock

Internamente, a máquina regista cada venda e envia alertas ao operador quando o stock está baixo ou quando há alguma avaria.

02

Sistemas da Máquina

Diferentes sistemas da composição

Estrutura e Gabinete

- Chassis/armário exterior
- Porta frontal de vidro

Sistema de Refrigeração

- Compressor
- Condensador e evaporador
- Ventoinhas
(Gera e circula o ar frio)

Sistema de Iluminação

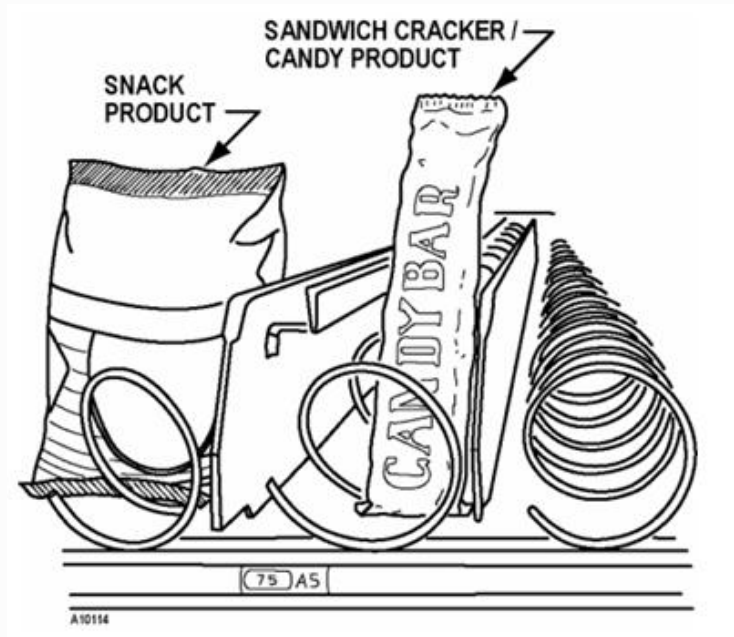
- Iluminação interior

Sistema de Pagamento

- Recetor de moedas
(valida e classifica moedas por tamanho e peso)
- Mecanismo de troco
(armazena e devolve moedas)

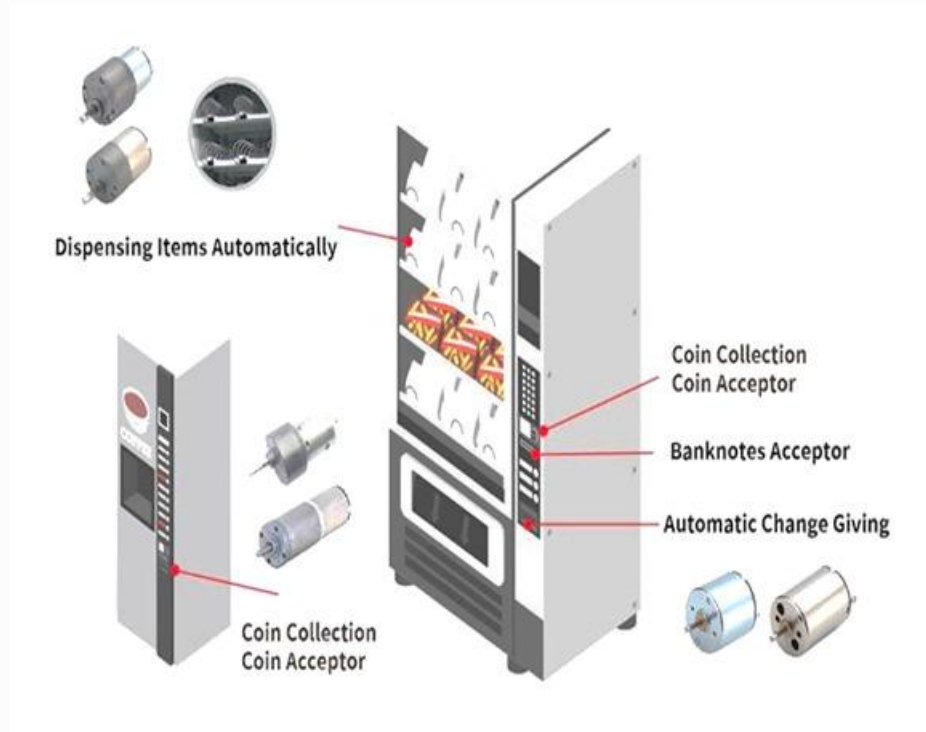
Sistema de Armazenamento e Dispensação

- Espirais motorizadas (empurram os produtos para a frente)
- Prateleiras ajustáveis (organizam os produtos)
- Bandeja de recolha (onde o cliente retira o produto)



Sistema Elétrico

- Microcontrolador – STM32
(gere os subsistemas)
- Fonte de alimentação
- Ecrã/display
(mostra preços, seleções e mensagens ao utilizador)
- Teclado de seleção
(botões para a escolha do produto)
- Motores DC e motor AC
- Sensores



03

Motores e sensores

Sistema de Potência

Motor DC e Sensor de Posição

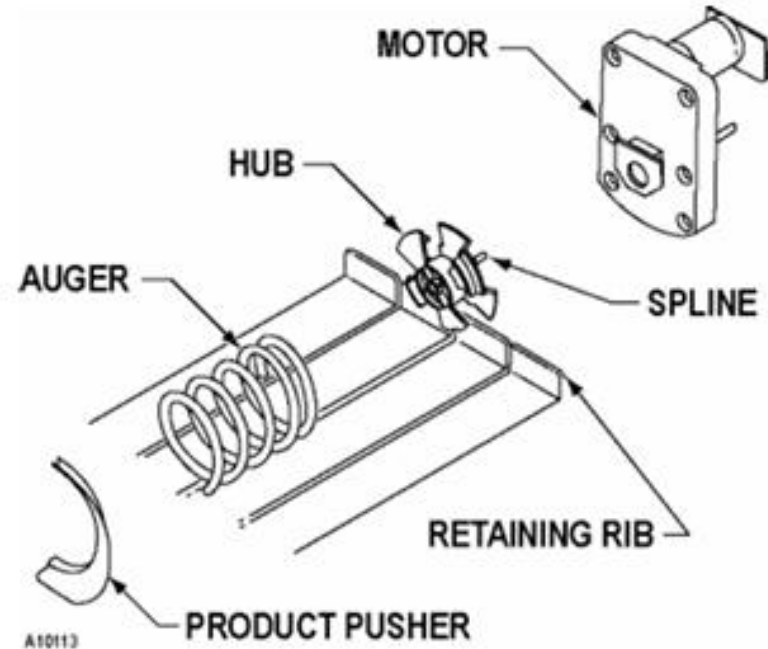
Motor DC (Corrente Contínua)

O microcontrolador envia um sinal elétrico ao motor quando o cliente seleciona um produto. O motor roda a espiral.

Sensor de Posição

Um sensor de posição deteta quando a espiral completou o ciclo e informa o microcontrolador para desligar o motor.

O produto é empurrado para fora da prateleira e cai por gravidade.



Motor AC

Para o sistema de refrigeração utiliza-se um motor de corrente alternada.

Mais robusto para este tipo de funcionamento contínuo e prolongado, em comparação com os motores DC que apenas são ativos pontualmente - arranques e paragens frequentes.

Os motores AC são construídos para dissipar calor de forma mais eficiente e têm menor desgaste em regime contínuo.

O compressor exige mais potência mecânica do que os outros motores da máquina. A rede elétrica AC (230V) fornece essa energia diretamente e de forma estável, sem precisar de conversão.

Sensor de Temperatura

O sensor de temperatura deteta se a temperatura atual está dentro dos limites e informa o microcontrolador para ligar ou desligar o motor da refrigeração.

Sensor Ótico

O emissor de infravermelho projeta um feixe de luz invisível de um lado e o recetor está do lado oposto, a 'receber' esse feixe. Quando o produto cai, interrompe o feixe por um instante, e assim informa o microcontrolador da queda.

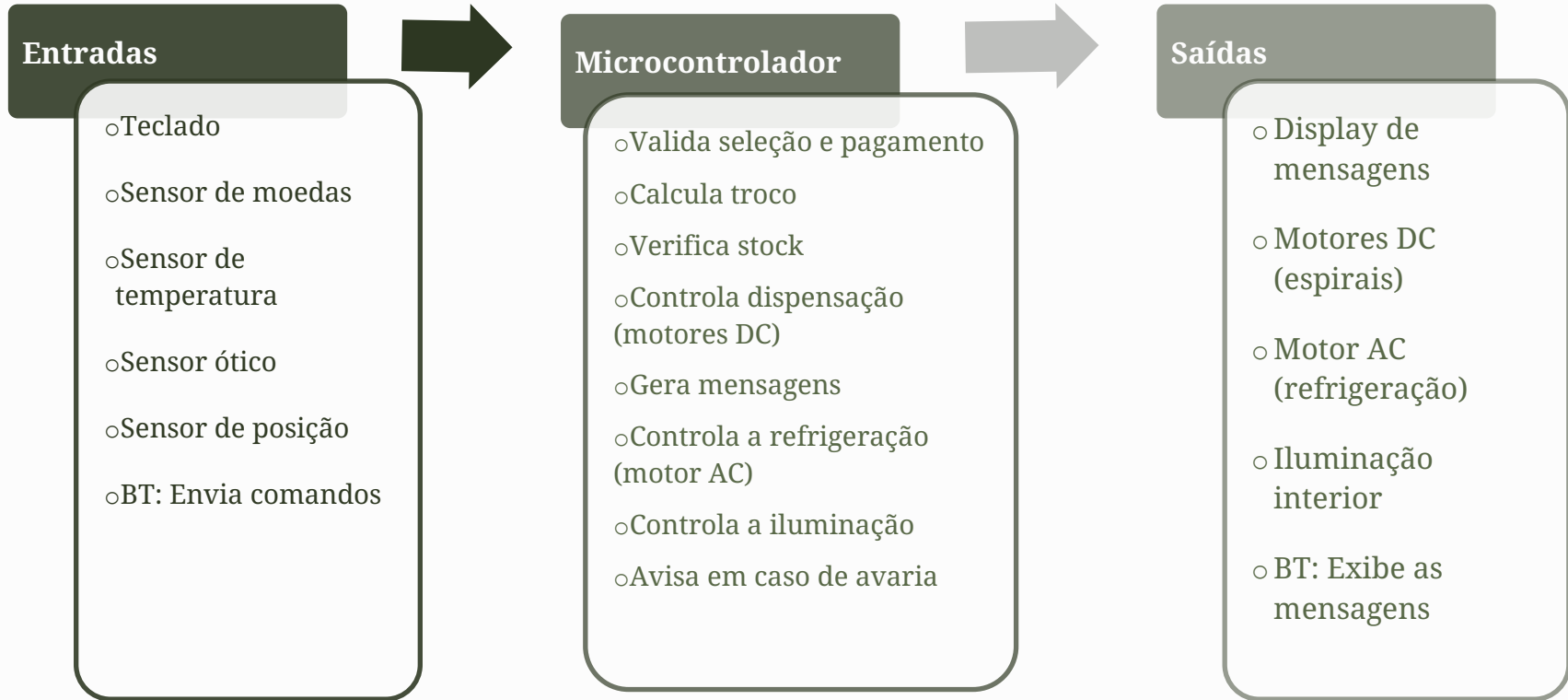


04

Microcontrolador

Sistema de Control

INPUTS & OUTPUTS



Esquema do Fluxo de Controlo

Controlo do Ambiente:

- Controlo da refrigeração
- Controlo da iluminação

Comunicação com o fornecedor:

- Estado de stock
- Alertas
- Temperatura
- Vendas

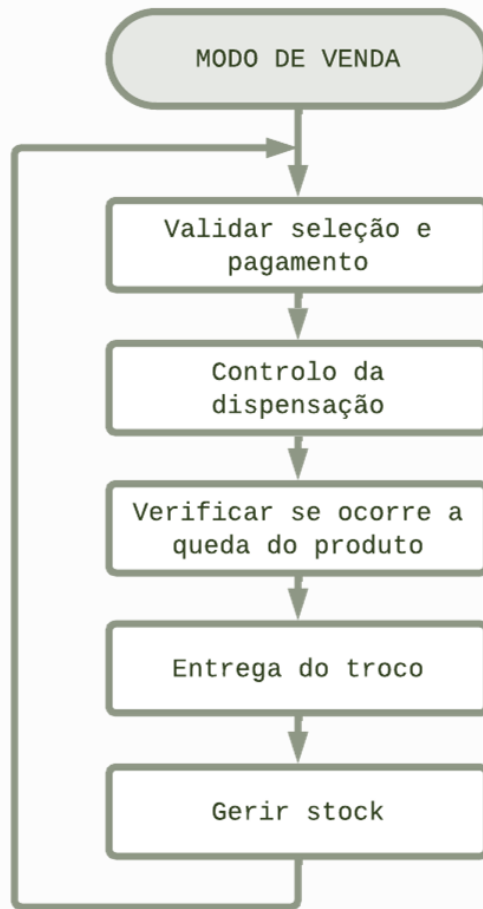
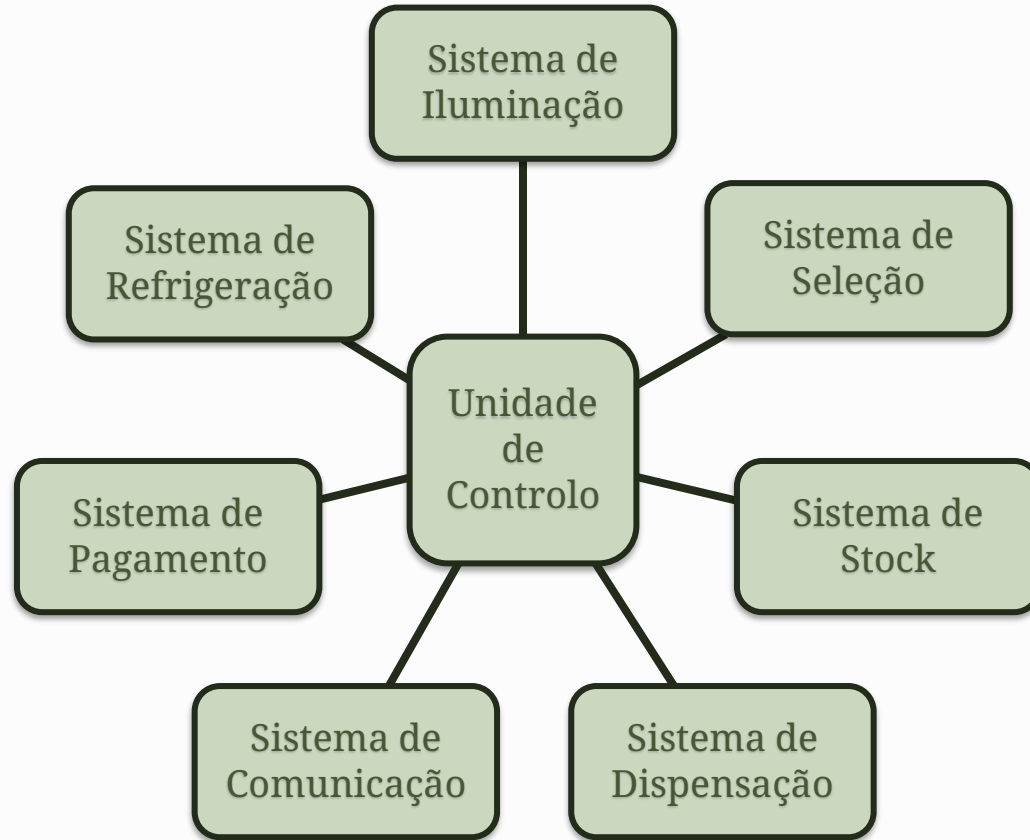


Diagrama De Blocos

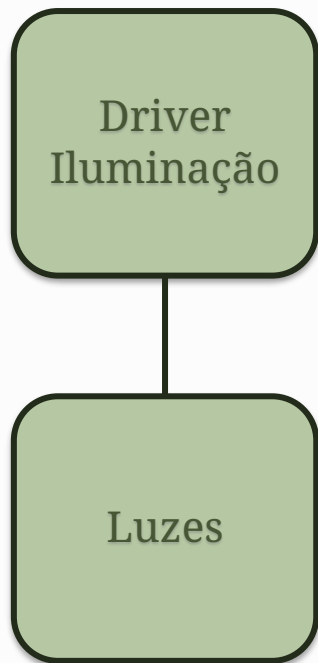


Design do diagrama de blocos

1. Sistema de Iluminação
 2. Sistema de Refrigeração
 3. Sistema de Pagamento
 4. Sistema de Seleção
 5. Sistema de Dispensação
 6. Sistema de Stock
 7. Sistema de Comunicação
- Especificação de sensores:
 - i. Temperatura
 - ii. Ótico
 - iii. Posição
 - iv. Teclado Numérico
 - Especificação de atuadores:
 - i. Display
 - ii. Motores DC e AC
 - iii. Luzes
 - Especificação de Comunicação

➤ MICROCONTROLADOR: STM32H755ZI-Q

Sistema de Iluminação



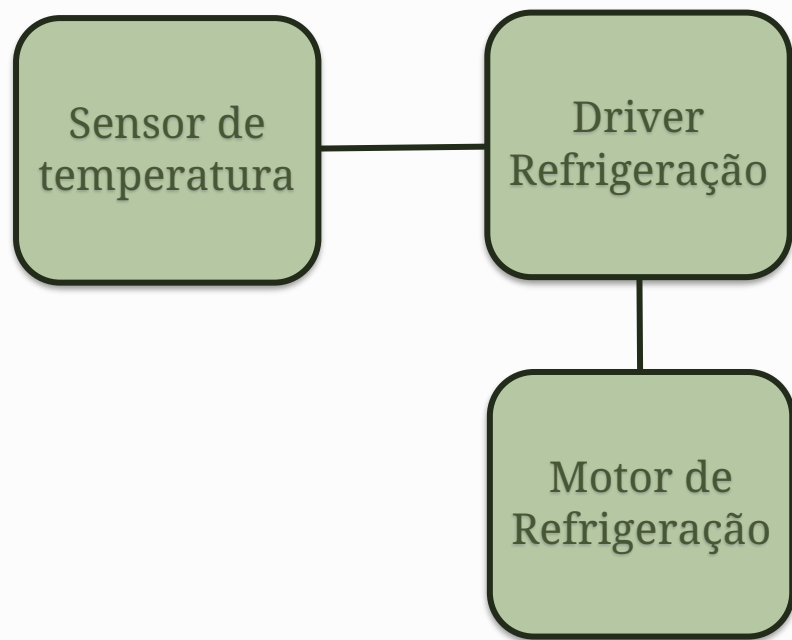
O controlo das luzes recorre ao **RTC** (*Real-Time Clock*) integrado na STM32.

Este módulo permite definir dois alarmes:

- o Alarme A, configurado para a hora de **ligar as luzes**,
- o Alarme B, configurado para a hora de **desligar as luzes**.

Quando a hora programada é atingida, o RTC dispara automaticamente uma **interrupção**, fazendo com que o microcontrolador pause momentaneamente a tarefa atual, execute a ação correspondente (ligar ou desligar as luzes) e retome de seguida o funcionamento normal.

Sistema de Refrigeração

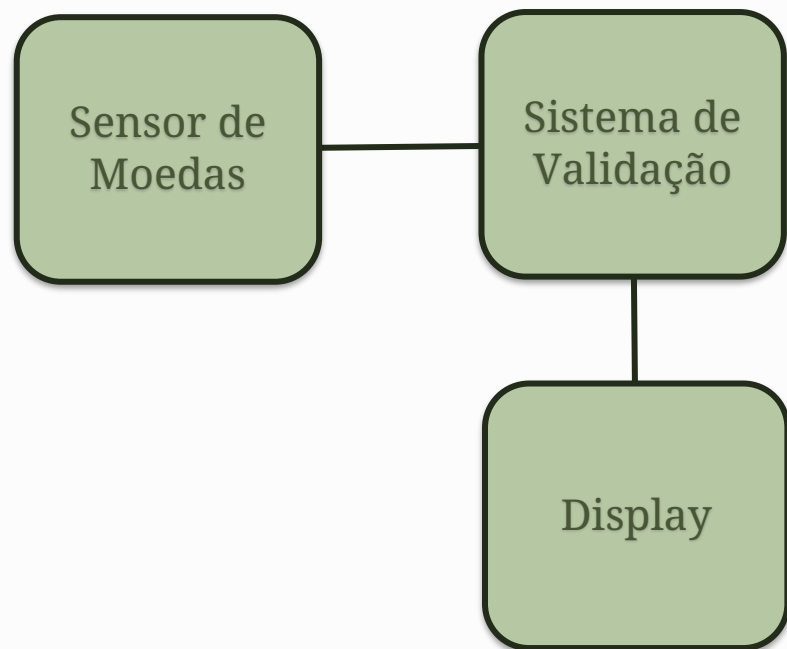


Com a informação fornecida pelo sensor, o microcontrolador verifica se a temperatura está dentro dos limites definidos.

Quando a temperatura fica acima do limite máximo programado, o microcontrolador liga o motor da refrigeração. Quando a temperatura volta ao intervalo adequado, o microcontrolador desliga o motor, poupando energia e mantendo os produtos conservados e em boas condições de consumo.



Sistema de Pagamento



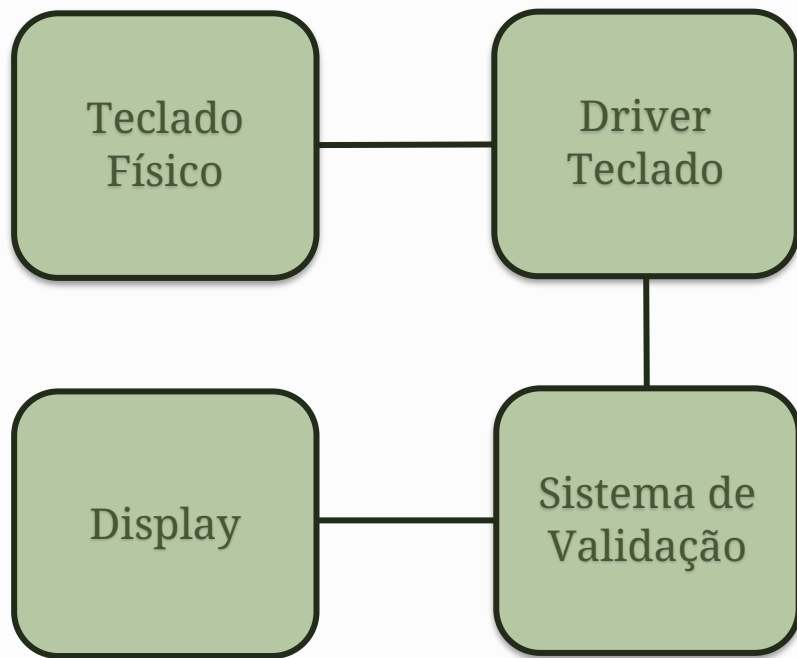
O utilizador introduz as moedas.

O sensor de moedas verifica o peso e tamanho, e o microcontrolador avalia se corresponde a um dos valores possíveis (moedas de 5 cêntimos a 2 euros).

O valor de saldo atual aparece no display.

Calcula e devolve o troco.

Sistema de Seleção



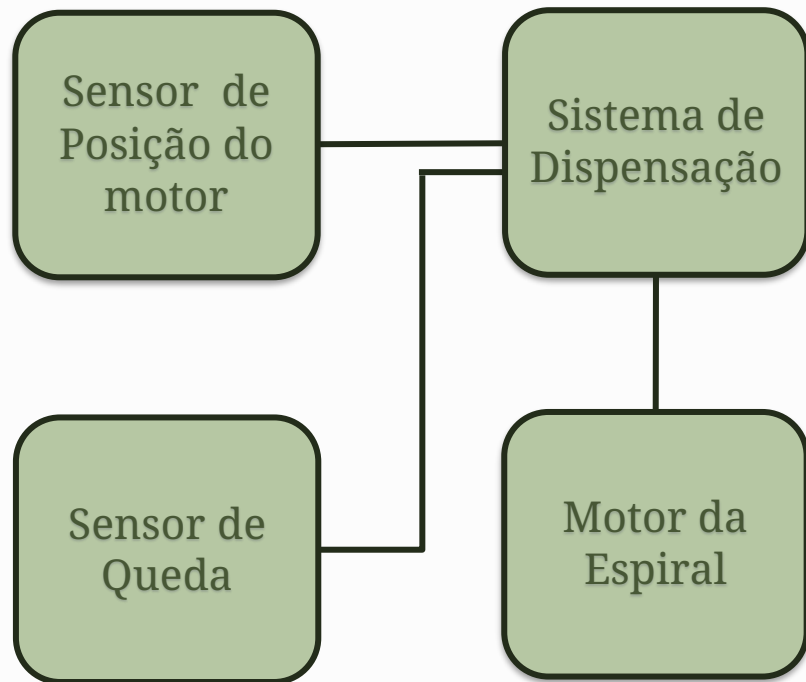
O utilizador faz a seleção do produto através do teclado físico.

O microcontrolador interpreta os sinais recebidos, e valida se pode aceitar essa seleção.

A interação entre o microcontrolador e o utilizador é disposta num display.

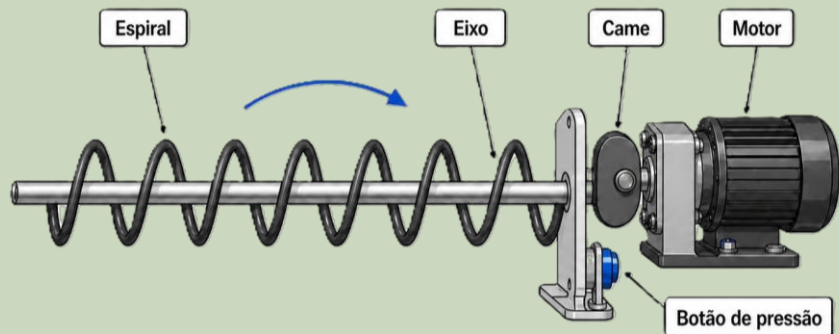
Uma vez que o teclado é constituído por botões de pressão, é necessário implementar a técnica de *debounce* para impedir seleções incorretas.

Sistema de Dispensação

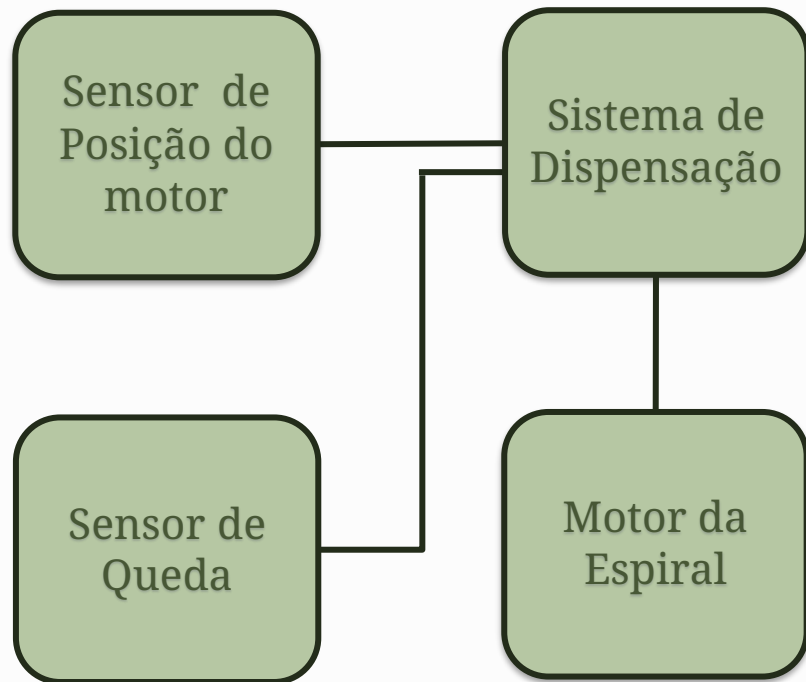


O motor da espiral selecionada é ligado.

O sensor de posição deteta quando a espiral completa o seu ciclo e envia um sinal ao microcontrolador para desligar o motor, evitando que continue a rodar.

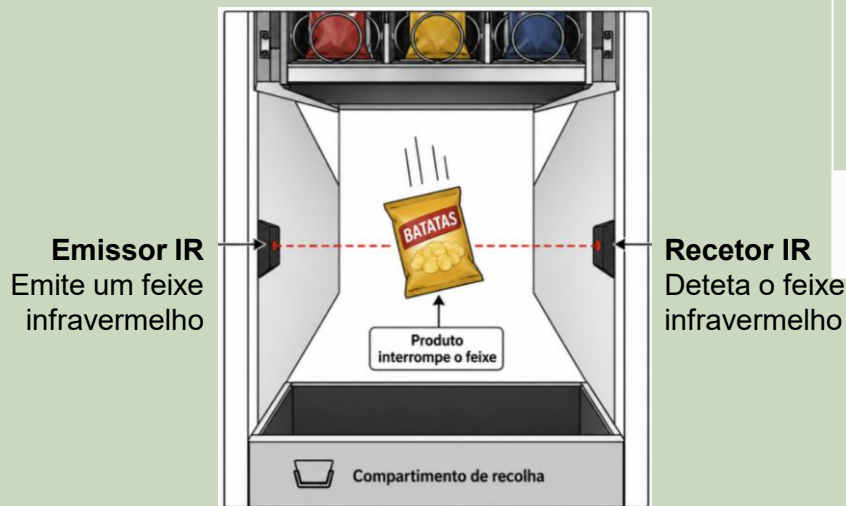


Sistema de Dispensação

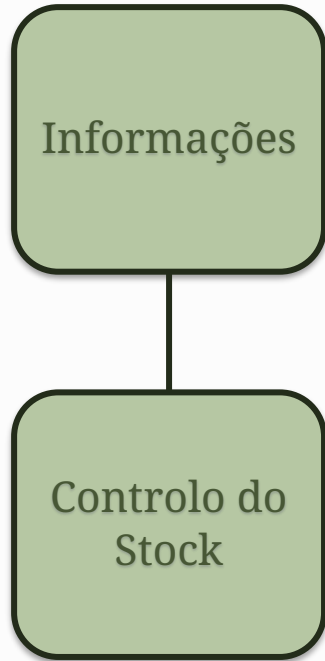


O sensor ótico verifica quando o produto cai, confirmando ao microcontrolador que o produto foi dispensado com sucesso.

A combinação dos dois sensores permite detetar falhas na entrega. Se a espiral completar o ciclo, mas o produto não cair, o sistema poderá rodar novamente, e avisar que houve uma falha na entrega.



Sistema de Stock



Cada vez que um produto é selecionado e posteriormente pago, o microcontrolador decrementa o contador interno desse produto.

Quando o stock de um produto atinge um valor mínimo pré-definido ou a sua validade está próxima de expirar, o sistema gera um alerta para o fornecedor.

Se o stock chegar a zero ou a validade expirar, o produto é marcado como indisponível e a seleção correspondente é bloqueada.

É mantida uma lista de informações sobre cada produto:

- Validade
- Preço
- Quantidade

Sistema de Comunicação

Comunicação
com
fornecedor

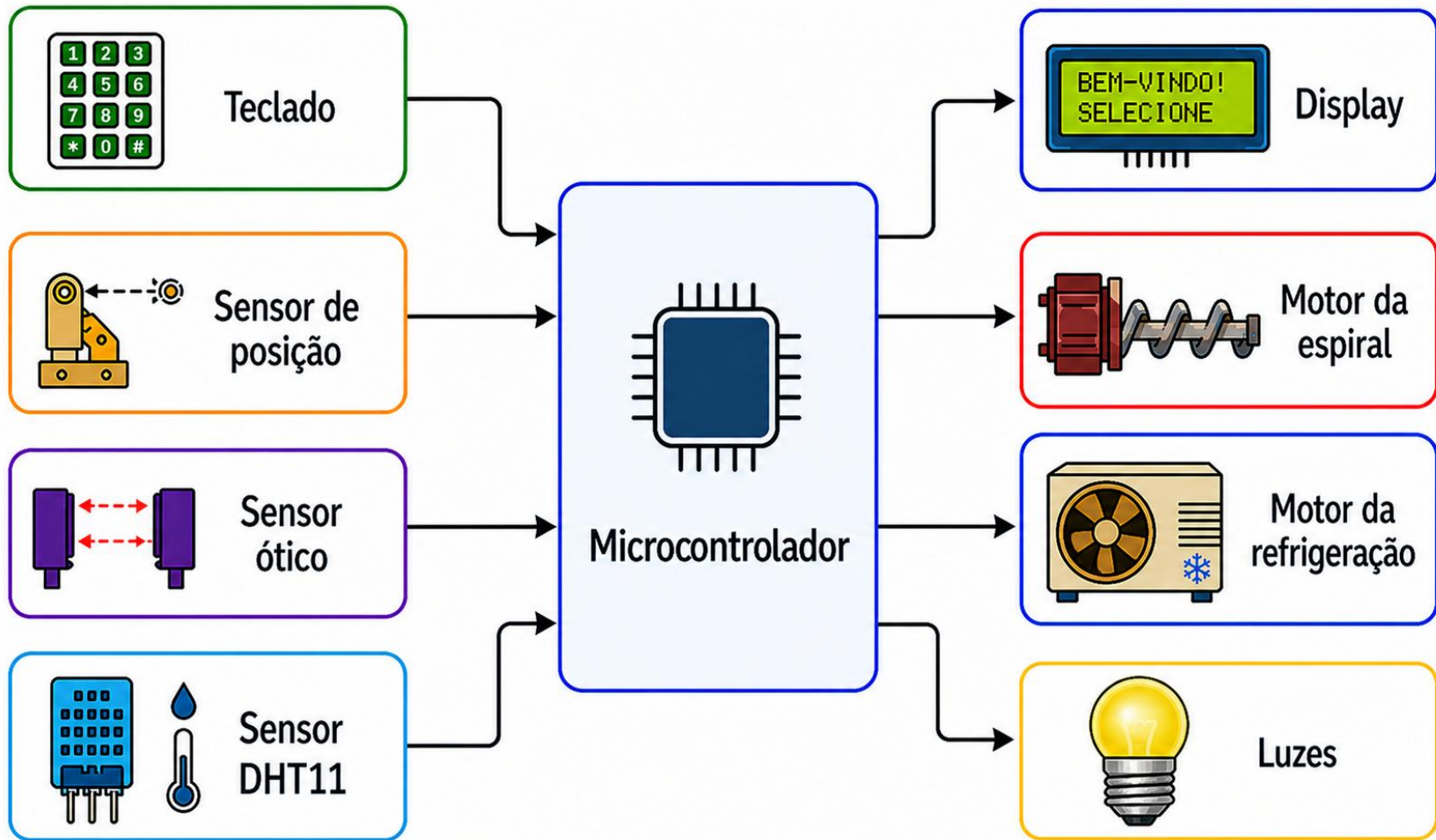
O microcontrolador pode enviar dados remotamente, permitindo ao operador monitorizar o estado da máquina (stock, avarias, vendas, ...) em tempo real.

Os eventos relevantes, como: falha na dispensação, stock baixo, temperatura fora dos limites, são registados e transmitidos.

Esta comunicação permite uma gestão mais eficiente, evitando deslocações desnecessárias para verificar o estado da máquina e um cuidado redobrado com a qualidade dos produtos.



Escolha De Material Para **Simulação**



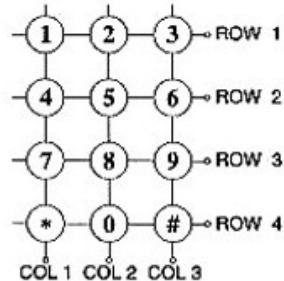
Teclado

O teclado possui 12 botões, que estão dispostas em 4 linhas por 3 colunas, e ele possui 7 pinos para ligação.

Cada interruptor em uma linha está conectado aos outros interruptores da mesma linha, assim como cada interruptor de uma coluna está conectado aos restantes dessa coluna.

Ou seja, todos os botões estão conectados a uma linha e a uma coluna.

CIRCUIT DIAGRAM



OUTPUT ARRANGEMENT

OUTPUT PIN NO.	SYMBOL
1	COL 2
2	ROW 1
3	COL 1
4	ROW 4
5	COL 3
6	ROW 3
7	ROW 2



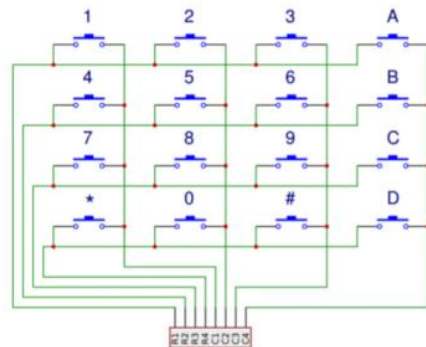
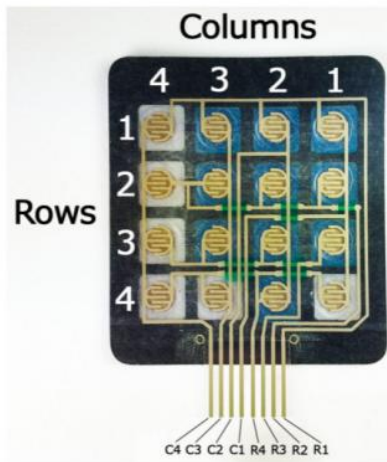
Sensor de Posição

Neste projeto, cada botão simula o sensor de posição de cada motor da espiral.

O matriz possui 16 botões, que estão dispostas em 4 linhas por 4 colunas, e ele possui 8 pinos para ligação.

Cada interruptor em uma linha está conectado aos outros interruptores da mesma linha, assim como cada interruptor de uma coluna está conectado aos restantes dessa coluna.

Ou seja, todos os botões estão conectados a uma linha e a uma coluna.



Matriz De Botões De Pressão 4x4

O microcontrolador deteta o botão premido ao detetar o pino de linha e coluna.

Nenhum botão premido:

1. Colunas: HIGH Linhas: LOW

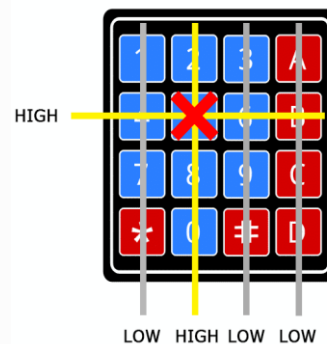
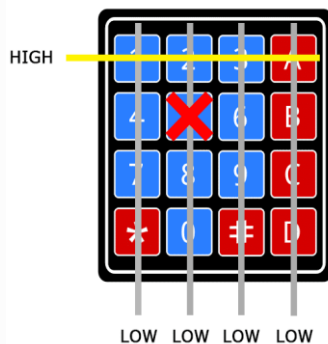
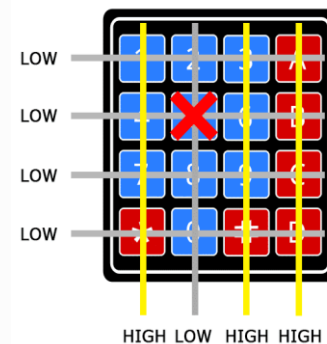
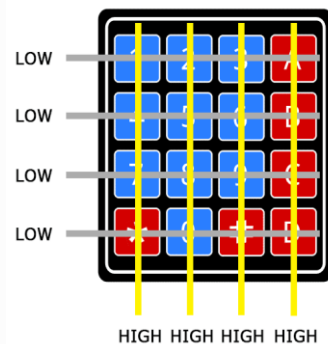
Botão premido:

2. O pino da coluna é puxado para LOW
(A tensão da coluna é igualada à da linha).

3. Colunas: LOW Linha: HIGH

4. O pino da linha é puxado para LOW
(A tensão da linha é igualada à da coluna).

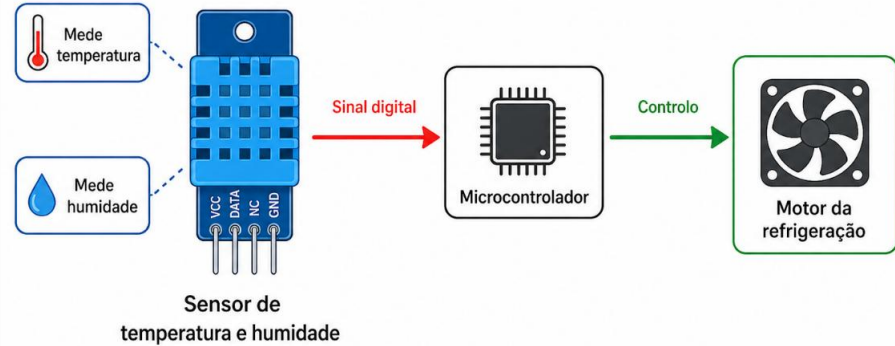
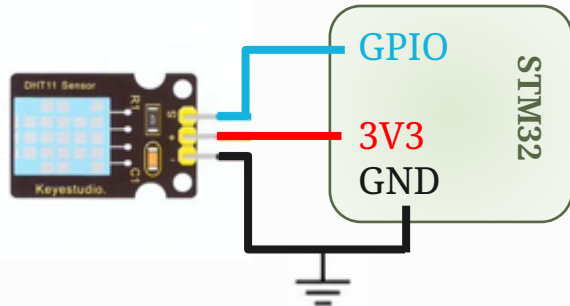
As etapas 3 e 4 são repetidas linha a linha até o microcontrolador encontrar a linha certa.



Sensor de Temperatura – DHT11

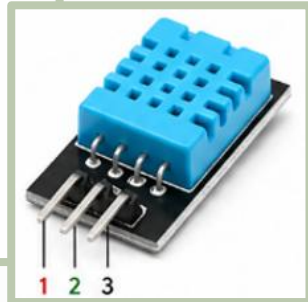
Sensor externo capaz de medir temperatura e humidade. Utiliza um termistor NTC (*Negative Temperature Coefficient*) para medir a temperatura e um componente resistivo para medir a humidade.

O próprio módulo faz o processamento da leitura e envia os dados ao microcontrolador através de um sinal digital (dispensa a conversão pelo ADC).



Características principais

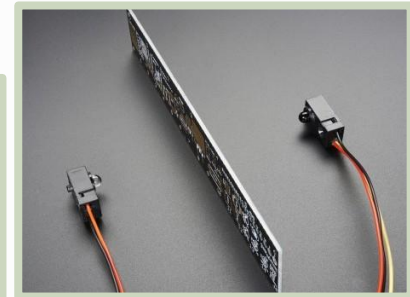
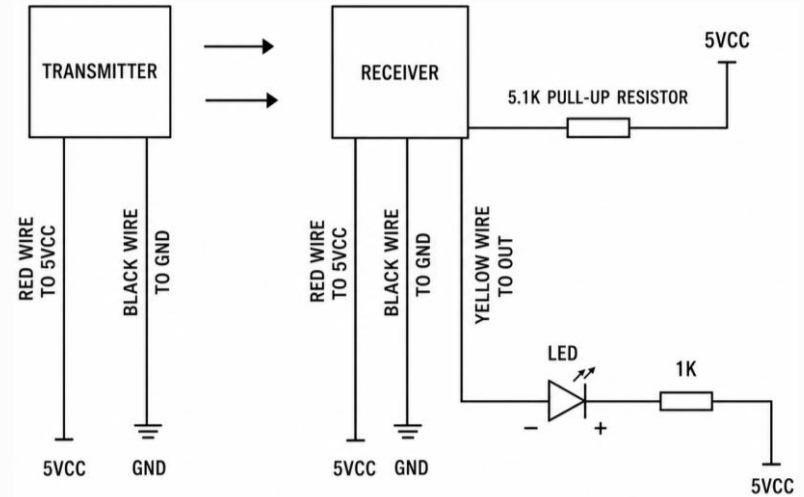
- Mede temperatura e humidade relativa do ar.
- Saída digital calibrada.
- Temperatura: 0 °C a 50 °C (precisão típica ± 2 °C).
- Humidade: 20% a 90% RH (precisão típica $\pm 5\%$ RH).
- Tensão de funcionamento: 3.3 V a 5.5 V.
- Comunicação por um único fio (protocolo próprio).



Sensor Ótico - IR

O sensor IR (Infrared) funcionam com um emissor que envia um feixe de luz infravermelha, e um recetor do outro lado que é sensível a essa mesma luz. Quando algo passa entre os dois e não é transparente ao IR, o feixe é interrompido e o recetor sinaliza.

É um sensor de 5 mm, pode funcionar com uma tensão de alimentação entre 3,3 V e 5 V, sendo recomendado 5 V para maior alcance. O emissor e o recetor podem trabalhar até cerca de 50 cm de distância. Como a saída do recetor é do tipo *open collector*, é necessário usar uma resistência *pull-up*, que pode ser interna no microcontrolador ou externa de 10 kΩ (entre o fio amarelo e o vermelho do recetor).



Emissor:

- Fio vermelho = 5VCC
- Fio preto = GND

Recetor:

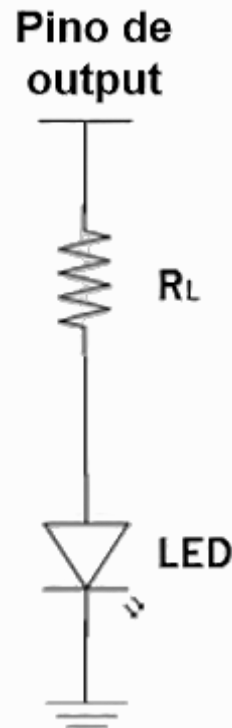
- Fio vermelho = 5VCC
- Fio preto = GND
- Fio amarelo = OUT (NPN)

Luzes

Neste projeto, o LED amarelo simula as luzes da máquina.

Motor de Refrigeração

Neste projeto, o LED azul simula o motor de refrigeração.



Cada um dos LEDs é controlado pelo respectivo pino de output da STM32.

Motor da Espiral

Neste projeto, cada LED simula cada motor da espiral.

A matriz é composta por 64 LEDs, dispostas em 8 linhas por 8 colunas, e ele possui 16 pinos para ligação.

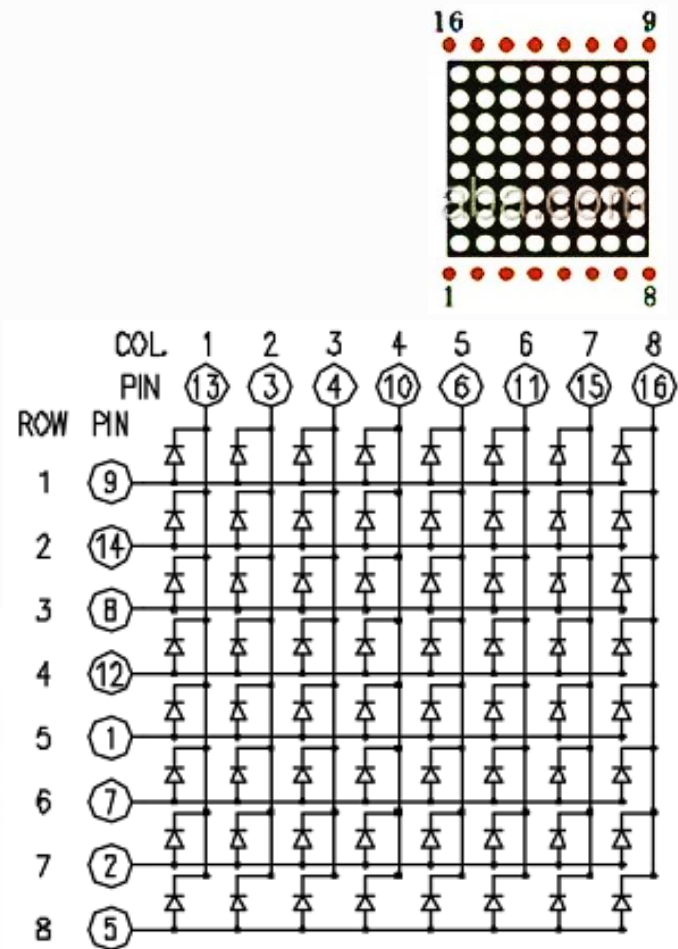
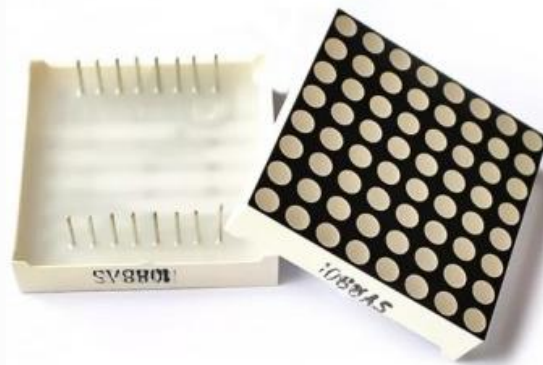
Quando uma linha está a HIGH e uma coluna a LOW, o LED correspondente acende.

É necessário utilizar resistências nos pinos que representam as colunas.

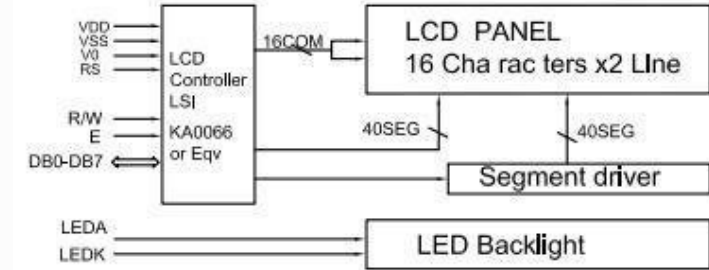
Especificações:

Tensão: 2.1 ~ 2.5 V

Corrente max: 20 mA.



Display - LCD1602



PIN NO.	SYMBOL	DESCRIPTION	FUNCTION
1	VSS	Ground	0V (GND)
2	VCC	Power Supply For Logic Circuit	+3.3V
3	V0	LCD Contrast Adjustment	Recommends 10K Potentiometer between V0 and GND
4	RS	Instruction/Data Register Selection	RS = 0 : Instruction Register RS = 1 : Data Register
5	R/W	Read/Write Selection	R/W = 0 : Register Write R/W = 1 : Register Read
6	E	Enable Signal	Active when High-level turns to Low-level
7 - 14	D0 – D7	Data Input/Output Lines	8 BIT: D0-D7
15	A	Supply Voltage For LED+	+3.3V
16	K	Supply Voltage For LED-	0V (GND)



Bluetooth

Para o Módulo de Bluetooth, foi escolhido o HC-05 para Arduino.

Para testar e utilizar esta comunicação, utilizamos a aplicação **Serial Bluetooth Terminal**, que permite estabelecer ligação Bluetooth com o módulo e enviar ou receber dados em formato série. A mesma possui dois códigos de desbloqueio distintos, que são o 1234 ou o 0000.

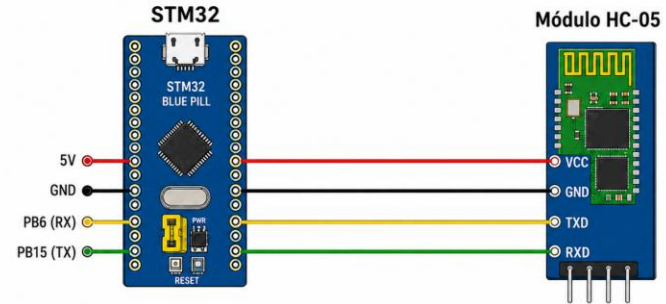


Características Principais:

- Comunicação Bluetooth clássica
- Baixo custo e tamanho reduzido
- Alimentação VCC de 5V e GND
- Ligações USART1: TXD → PB6 (RX) e RXD → PB15 (TX)

HC-05 com STM32

Ligações principais



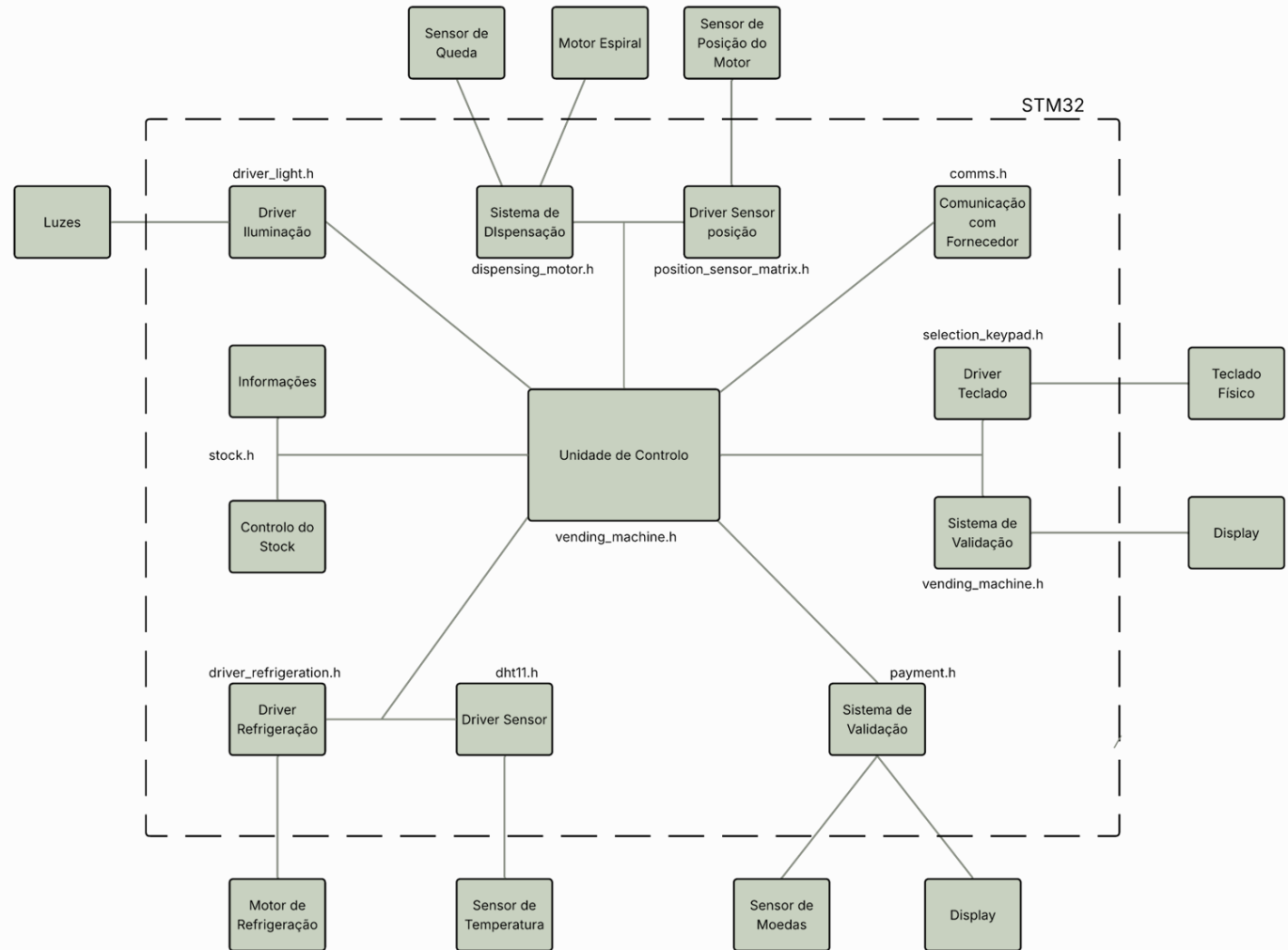
HC-05	STM32
VCC	5V
GND	GND
TXD	PB6 (RX)
RXD	PB15 (TX)

Funções e limitações:

Permite o controlo remoto e monitorização simples. O alcance é limitado e não é indicado para grandes volumes de dados.

Algoritmo e Interface do Programa

Instanciação do sistema nas plataformas computacionais



MAIN

INIT:

- init GPIO, RTC, TIM3, TIM6, UART1, UART3
- init luzes (driver_light)
- init refrigeração (TIM3 para DHT11)
- iniciar TIM6 com interrupção a cada 2s
- init teclado, LCD, vending machine

LOOP:

- Refrig_Update()
- VM_Update()

INTERRUPÇÕES:

- UART3 RX → VM_UartRxCallback() // moedas
- UART1 RX → Comms_RxCallback() // Bluetooth
- TIM6 → hrefrig.ReadPending=1 // trigger leitura DHT11

VENDING_MACHINE

VM_Update():

SE price>0 E balance=0 E timeout expirou → VM_Goldle()

SE idle E balance=0 E price=0 E timeout → rodar ecrã idle

SE moeda inserida → mostrar saldo

processar comandos Bluetooth

ler tecla do teclado:

'*' → VM_DoCancel()

1º dígito válido (1-4) → guardar, estado=SELECAO

2º dígito válido (1-4) → VM_TrySelect()

dígito inválido → mostrar erro 1.5s

VM_TrySelect():

SE VM_IsValidSelection() falhar → limpar seleção, return

verificar stock (EMPTY/EXPIRED → erro 2s; EXPIRY_SOON → aviso 1.5s)

definir preço

SE balance >= price → VM_DoDispensing()

SENÃO → mostrar price e balance

VM_DoDispensing():

calcular troco

executar motor (Dispensing_Execute)

SE sucesso → decrementar stock, "Thank you!", delay 2s

SE falha → "Dispense error!", delay 3s

Payment_Reset(), VM_Goldle()

VM_DoCancel():

mostrar "Cancelled!" + troco, delay 2s

Payment_Reset(), VM_Goldle()

VENDING_MACHINE. h

```
#ifndef VENDING_MACHINE_H
#define VENDING_MACHINE_H

#include "stm32h7xx_hal.h"
#include "main.h"
#include "lcd.h"
#include "selection_keypad.h"
#include "payment.h"
#include "stock.h"
#include "comms.h"
#include "driver_refrigeration.h"
#include <stdio.h>

#define VM_IDLE_ROTATE_MS 2000
#define VM_PRICE_TIMEOUT_MS 5000

typedef enum {
    VM_IDLE ,
    VM_SELECAO,
    VM_DISPENSING,
} VM_State;
```

```
typedef struct {
    VM_State state;
    char product[3];
    uint8_t count;
    Keypad_HandleTypeDef *hkp;
    Keypad_HandleTypeDef *hsensor;
    Payment_HandleTypeDef payment;
    Stock_HandleTypeDef stock;
    uint8_t idle_screen;
    uint32_t idle_last_tick;
    uint32_t price_shown_tick;
    uint16_t total_sales;
    uint32_t total_revenue;
} VM_HandleTypeDef;

void VM_Init(VM_HandleTypeDef *hvm, Keypad_HandleTypeDef *hkp,
Keypad_HandleTypeDef *hsensor);
void VM_Update(VM_HandleTypeDef *hvm);
void VM_UartRxCallback(VM_HandleTypeDef *hvm);

#endif
```

PAYMENT

Payment_Update():

SE linha UART3 completa:

parsear "weight diameter thickness"

identificar moeda (comparar com tabela $\pm$ tolerância)

SE reconhecida $\rightarrow$ balance += valor, return 1

SENÃO $\rightarrow$ reportar erro, return 0

Payment_RxCallback():

acumular bytes até '\n' $\rightarrow$ lineReady = 1

Payment_Reset():

enviar "Change returned: X.XX EUR" via UART3

balance = 0, price = 0

Tabela de moedas: 5cent, 10cent, 20cent, 50cent, 1€, 2€

identificadas por peso, diâmetro e espessura

PAYMENT.h

```
#ifndef PAYMENT_H
```

```
#define PAYMENT_H
```

```
#include "stm32h7xx_hal.h"
```

```
#include "usart.h"
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define COIN_TOLERANCE 0.10f
```

```
typedef struct {uint16_t price; uint16_t balance; uint8_t rxByte;
```

```
char rxLine[32]; uint8_t rxIdx; uint8_t lineReady;
```

```
} Payment_HandleTypeDef;
```

```
void Payment_Init(Payment_HandleTypeDef *hp);
```

```
void Payment_SetPrice(Payment_HandleTypeDef *hp, uint16_t price);
```

```
void Payment_StartRx(Payment_HandleTypeDef *hp);
```

```
void Payment_RxCallback(Payment_HandleTypeDef *hp);
```

```
uint8_t Payment_Update(Payment_HandleTypeDef *hp);
```

```
void Payment_Reset(Payment_HandleTypeDef *hp);
```

```
#endif
```

STOCK

Stock_Init():

copiar tabela inicial para hs->products
verificar validades e enviar alertas BT

Stock_Get(row, col):

SE qty=0 → EMPTY
SE expirado → EXPIRED
SE expira em breve → EXPIRY_SOON
SE qty <= mínimo → LOW
SENÃO → OK

Stock_Decrement(row, col):

qty--
SE qty=0 → alerta BT "vazio"
SE qty <= mínimo → alerta BT "baixo"

Stock_CheckExpiry():

obter data hoje do RTC
diff = dias(validade) - dias(hoje)
SE diff < 0 → EXPIRED
SE diff <= ALERT_DIAS → EXPIRY_SOON
SENÃO → OK

STOCK. h

```
#ifndef STOCK_H
#define STOCK_H
#include "stm32h7xx_hal.h"
#include "rtc.h"
#include "comms.h"
#include <stdio.h>
#define STOCK_MIN_ALERT    2
#define STOCK_EXPIRY_ALERT  7
#define STOCK_ROWS        4
#define STOCK_COLS        4

typedef struct {uint8_t day; uint8_t month;uint8_t year;} Stock_Date;
typedef struct {uint8_t qty; uint16_t price; Stock_Date expiry;
} Stock_Product;

static const Stock_Product StockInitial[STOCK_ROWS][STOCK_COLS] = {
    {{ 5, 50, {30,12,26} }, { 5, 50, {15, 1,27} }, { 5, 75, {20, 6,27} }, { 5, 75, { 1, 3,27} }},
    {{ 5, 75, {10, 7,27} }, { 5, 100, {25, 8,27} }, { 5, 100, { 5, 6,26} }, { 5, 125, {30, 9,27} }},
    {{ 5, 100, { 1,11,27} }, { 5,125, {15,12,27} }, { 5, 150, {20, 1,28} }, { 5, 150, {10, 2,28} }},
    {{ 5, 125, { 5, 3,28} }, { 5, 150, { 1, 4,28} }, { 5, 175, {15, 5,28} }, { 5, 200, {30, 6,28} }}

};
```

STOCK. h

```
typedef enum { STOCK_OK ,  
               STOCK_LOW,  
               STOCK_EMPTY,  
               STOCK_EXPIRED,  
               STOCK_EXPIRY_SOON,  
} Stock_StatusTypeDef;
```

```
typedef struct {  
    Stock_Product products[STOCK_ROWS][STOCK_COLS];  
} Stock_HandleTypeDef;
```

```
void Stock_Init(Stock_HandleTypeDef *hs);  
Stock_StatusTypeDef Stock_Get(Stock_HandleTypeDef *hs, uint8_t row, uint8_t col);  
Stock_StatusTypeDef Stock_Decrement(Stock_HandleTypeDef *hs, uint8_t row, uint8_t col);  
uint16_t Stock_GetPrice(Stock_HandleTypeDef *hs, uint8_t row, uint8_t col);  
void Stock_SendStatus(Stock_HandleTypeDef *hs);  
void Stock_SendStatusProduto(Stock_HandleTypeDef *hs, uint8_t row, uint8_t col);  
uint8_t Stock_SetQty(Stock_HandleTypeDef *hs, uint8_t row, uint8_t col, uint8_t qty);  
uint8_t Stock_SetExpiry(Stock_HandleTypeDef *hs, uint8_t row, uint8_t col, uint8_t day, uint8_t month, uint8_t year);  
  
#endif
```

COMMS

Comms_RxCallback(): // UART1 - Bluetooth

acumular bytes até '\n'

parsear comando:

"STOCK:STATUS"	→ flag stockStatusRequested
"TEMP"	→ flag tempRequested
"VENDAS"	→ flag vendasRequested
"AJUDA"	→ flag ajudaRequested
"STOCK:LxCx"	→ flag stockQueryReady
"STOCK:LxCx:QTY:n"	→ flag setQtyReady
"STOCK:LxCx:VAL:DD/MM/YY"	→ flag setValReady

Funções de envio BT (todas com timestamp RTC):

SendVenda, SendStockBaixo, SendStockVazio,
SendValidadeProxima, SendValidadeExpirada,
SendDispensacaoOK, SendDispensacaoFalha,
SendTempAlta, SendTempCritica, SendTempNormal,
SendSensorDHT11Erro, SendArranque, SendAjuda

COMMS. h

```
#ifndef COMMS_H
#define COMMS_H
#include "stm32h7xx_hal.h"
#include "usart.h"
#include "rtc.h"
#include <stdio.h>
#include <string.h>
```

```
extern UART_HandleTypeDef huart1;
void Comms_Send(const char *msg);
void Comms_StartRx(void);
void Comms_RxCallback(void);
```

```
uint8_t Comms_IsStockStatusRequested(void);
uint8_t Comms_IsTempRequested(void);
uint8_t Comms_IsVendasRequested(void);
uint8_t Comms_IsAjudaRequested(void);
```

```
uint8_t Comms_IsStockQueryRequested
(uint8_t *row, uint8_t *col);
```

```
uint8_t Comms_IsStockSetQtyRequeste
(uint8_t *row, uint8_t *col, uint8_t *qty);
```

```
uint8_t Comms_IsStockSetValRequested(uint8_t *row, uint8_t *col, uint8_t *day,
uint8_t *month, uint8_t *year);
```

```
void Comms_SendArranque(void);
void Comms_SendAjuda(void);
```

```
void Comms_SendTempActual(float temp, float humidity);
void Comms_SendVendas(uint16_t total_sales, uint32_t total_revenue);
```

```
void Comms_SendVenda(uint8_t row, uint8_t col, uint16_t price, uint16_t change);
void Comms_SendStockBaixo(uint8_t row, uint8_t col, uint8_t qty);
void Comms_SendStockVazio(uint8_t row, uint8_t col);
void Comms_SendValidadeProxima(uint8_t row, uint8_t col, int32_t days, uint8_t
day, uint8_t month, uint8_t year);
void Comms_SendValidadeExpirada(uint8_t row, uint8_t col, uint8_t day, uint8_t
month, uint8_t year);
void Comms_SendDispensacaoOK(uint8_t row, uint8_t col);
void Comms_SendDispensacaoFalha(uint8_t row, uint8_t col);
void Comms_SendTempAlta(float temp, float limit);
void Comms_SendTempCritica(float temp);
void Comms_SendTempNormal(float temp);
void Comms_SendSensorDHT11Erro(uint8_t status_code);
```

```
#endif
```

DISPENSING_MOTOR

Dispensing_Execute(row, col):
SE IR já bloqueado (produto no tabuleiro):
ligar motor(row, col)
aguardar sensor posição (espiral completa) até timeout
→ SUCCESS se sensor ativo, FAILURE se timeout

SENÃO (IR livre), até 2 ciclos:
ligar motor
aguardar:
IR bloqueado → SUCCESS (produto caiu)
sensor posição ativo (só ciclo 1) → SUCCESS
timeout → parar, pausar, tentar ciclo 2
IR bloqueado → SUCCESS (produto caiu)
após 2 ciclos sem sucesso → FAILURE

Motor: matriz LED 4x4 (row HIGH + col LOW = motor ON)

DISPENSING_MOTOR. h

```
#ifndef DISPENSING_MOTOR_H
#define DISPENSING_MOTOR_H

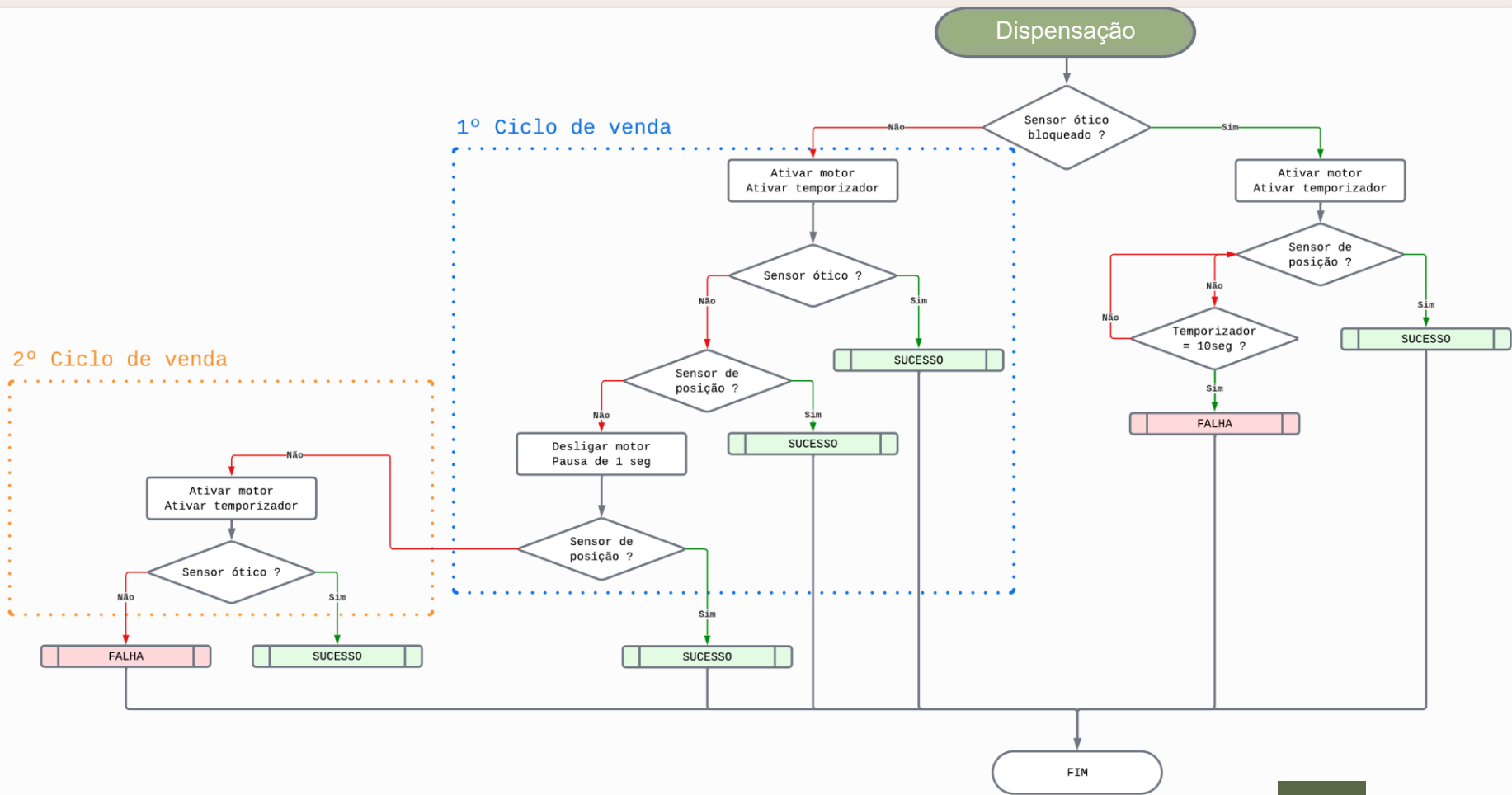
#include "stm32h7xx_hal.h"
#include "main.h"
#include "driver_button_matrix.h"
#include "lcd.h"

#define SPIRAL_TIMEOUT_MS 10000
#define SPIRAL_PAUSE_MS 1000

void Dispensing_Init(void);

uint8_t Dispensing_Execute(Keypad_HandleTypeDef *hsensor,
uint8_t row, uint8_t col, char line_char, char col_char);

#endif
```



DHT11. h

```
#ifndef DHT11_H
#define DHT11_H
#include "stm32h7xx_hal.h"
#include "stm32h7xx_hal_tim.h"
```

```
typedef enum {
    DHT11_OK ,
    DHT11_TIMEOUT,
    DHT11_ERROR,
    DHT11_CHECKSUM_MISMATCH,
    __DHT11_STATUS_TYPEDEF_COUNT__
} DHT11_StatusTypeDef;
```

```
typedef struct {
    float Temperature;
    float Humidity;
    DHT11_StatusTypeDef Status;
    uint16_t _Pin;
    GPIO_TypeDef *_GPIOx;
    TIM_HandleTypeDef *_Tim;
} DHT11_InitTypeDef;
```

```
void HAL_DHT11_Init(DHT11_InitTypeDef *DHT11, GPIO_TypeDef *GPIOx,
                    uint16_t GPIO_Pin, TIM_HandleTypeDef *TIM);
```

```
DHT11_StatusTypeDef HAL_DHT11_ReadData(DHT11_InitTypeDef *DHT11);
```

```
void HAL_DHT11_DeInit(DHT11_InitTypeDef *DHT11);
```

```
const char* const HAL_DHT11_GetErrorMsg(DHT11_StatusTypeDef Status);
```

```
float HAL_DHT11_ReadTemperatureC(DHT11_InitTypeDef *DHT11);
```

```
float HAL_DHT11_ReadTemperatureF(DHT11_InitTypeDef *DHT11);
```

```
float HAL_DHT11_ReadHumidity(DHT11_InitTypeDef *DHT11);
```

```
#endif
```

DRIVER_LIGHT

driver_light_init():

ler hora do RTC

SE hora entre LIGHTS_ON e LIGHTS_OFF → ON

SENÃO → OFF

Alarme A RTC (07:00) → ligar luzes

Alarme B RTC (20:00) → apagar luzes

DRIVER_LIGHT.h

```
#ifndef DRIVER_LIGHT_H
```

```
#define DRIVER_LIGHT_H
```

```
#include "stm32h7xx_hal.h"
```

```
#include "rtc.h"
```

```
#include "gpio.h"
```

```
#define LIGHTS_ON_HOUR 7
```

```
#define LIGHTS_OFF_HOUR 20
```

```
extern void driver_light_init(void);
```

```
#endif
```

DRIVER_BUTTON_MATRIX SELECTION_KEYPAD POSITION_SENSOR_MATRIX

Keypad_Read():
para cada row:
colocar row LOW, restantes HIGH
para cada col:
SE col LOW → tecla pressionada
debounce, aguardar soltar tecla, debounce
→ retornar keyMap[row][col]
→ retornar 0 (nenhuma tecla)

selection_keypad: mapa 4x3 → '1'-'9', '*', '0', '#'

position_sensor: mapa 4x4 → valores 1-16 (sensores das espirais)

DRIVER_BUTTON_MATRIX. h

```
#ifndef DRIVER_BUTTON_MATRIX_H  
#define DRIVER_BUTTON_MATRIX_H
```

```
#include "stm32h7xx.h"
```

```
#define KEYPAD_MAX_ROWS 4  
#define KEYPAD_MAX_COLS 4  
#define KEYPAD_DEBOUNCE_MS 20
```

```
typedef struct {  
    GPIO_TypeDef *rowPorts[KEYPAD_MAX_ROWS];  
    uint16_t rowPins[KEYPAD_MAX_ROWS];  
    GPIO_TypeDef *colPorts[KEYPAD_MAX_COLS];  
    uint16_t colPins[KEYPAD_MAX_COLS];  
    uint8_t rows;  
    uint8_t cols;  
    const char (*keyMap)[KEYPAD_MAX_COLS];  
} Keypad_HandleTypeDef;
```

```
void Keypad_Init(Keypad_HandleTypeDef *hkp);  
char Keypad_Read(Keypad_HandleTypeDef *hkp);
```

```
#endif
```

SELECTION_KEYPAD. h

```
#ifndef SELECTION_KEYPAD_H
#define SELECTION_KEYPAD_H

#include "driver_button_matrix.h"

extern Keypad_HandleTypeDef hkp_selecao;

#endif
```

POSITION_SENSOR_MATRIX. h

```
#ifndef POSITION_SENSOR_MATRIX_H
#define POSITION_SENSOR_MATRIX_H

#include "driver_button_matrix.h"

extern Keypad_HandleTypeDef hsensor;

#endif
```

LCD

lcd_init():

sequência HD44780 em modo 4-bit

configurar: 2 linhas, fonte 5x8, cursor OFF

lcd_send_cmd(cmd): RS=LOW, high nibble + low nibble

lcd_send_data(c): RS=HIGH, high nibble + low nibble

lcd_put_cur(row,col): definir posição do cursor

lcd_clear(): limpar o ecrã

lcd_send_string(str): enviar caracter a caracter

LCD. h

```
#ifndef LCD_H
```

```
#define LCD_H
```

```
#include "stm32h7xx_hal.h"
```

```
void lcd_init(void);
```

```
void lcd_clear(void);
```

```
void lcd_put_cur(int row, int col);
```

```
void lcd_send_string(char *str);
```

```
void lcd_send_data(char data);
```

```
void lcd_send_cmd(char cmd);
```

```
#endif
```

DRIVER_REFRIGERATION

```
Refrig_Init():
    init DHT11 (TIM3 para delays µs)
    estado = HEATING, motor arrefecimento OFF

// chamado quando ReadPending=1 (TIM6 a cada 2s)
Refrig_Update():
    ler DHT11
    SE erro → alerta BT, return
    SE temp > CRITICO → cooling ON, alerta crítico
    SE temp > ON (7°C) → cooling ON, alerta (só na transição)
    SE temp < OFF (3°C) → cooling OFF, info (só na transição)

// Histerese: liga a 7°C, desliga a 3°C
```

DRIVER_REFRIGERATION. h

```
#ifndef DRIVER_REFRIGERATION_H
#define DRIVER_REFRIGERATION_H
#include "DHT11.h"
#include "stm32h7xx_hal.h"
#include "gpio.h"
#include "comms.h"

#define REFRIG_TEMP_REFERENCE 5.0f
#define REFRIG_HYSTERESIS 2.0f
#define REFRIG_TEMP_CRITICAL 15.0f

typedef enum {REFRIG_HEATING , REFRIG_COOLING
} Refrig_StateTypeDef;
typedef struct {DHT11_InitTypeDef Sensor; Refrig_StateTypeDef State;
float LastTemperature; float LastHumidity;
DHT11_StatusTypeDef LastStatus; volatile uint8_t ReadPending;
} Refrig_HandleTypeDef;
extern Refrig_HandleTypeDef hrefrig;

void Refrig_Init(Refrig_HandleTypeDef *hrefrig, TIM_HandleTypeDef
*htim);
void Refrig_Update(Refrig_HandleTypeDef *hrefrig);

#endif
```

Cenários de Teste

PAGAMENTO - UART3

normal fluxo esperado

erro entrada inválida ou falha

limite caso extremo

auto comportamento automático

normal

Inserir moeda válida

Enviar "3.92 21.25 1.67" → reconhece 5 centimos, saldo atualizado no LCD

normal

Inserir várias moedas

Saldo acumula corretamente com múltiplas moedas até cobrir o preço

erro

Moeda não reconhecida

Medições fora da tolerância → "Not recognized" via UART3, saldo inalterado

erro

Formato inválido UART

Enviar texto malformato → "Invalid format" via UART3, sem crash

normal

Troco devolvido

Após compra ou cancelamento, troco enviado via UART3 e saldo zerado

SELEÇÃO DO PRODUTO - TECLADO

normal fluxo esperado

erro entrada inválida ou falha

limite caso extremo

auto comportamento automático

normal

Seleção válida com saldo

Inserir "1" depois "2" com saldo suficiente → motor arranca

normal

Seleção sem saldo suficiente

Preço mostrado no LCD, aguarda mais moedas durante VM_PRICE_TIMEOUT_MS

auto

Timeout sem saldo

Sem moedas inseridas, ecrã de preço desaparece após VM_PRICE_TIMEOUT_MS e regressa ao idle

erro

Dígito fora do intervalo

Teclas '0' ou '5'-'9' → "Invalid! (1-4)" no LCD, 1.5s, sem alterar saldo

erro

Produto esgotado

"Out of stock!" no LCD, 2s, regressa ao ecrã anterior

erro

Produto expirado

"Prod. Expired!" no LCD, 2s, regressa ao ecrã anterior

limite

Produto a expirar

"Expiring soon!" 1.5s mas permite a compra na mesma

normal

Cancelar com '**

Em qualquer momento → "Cancelled!" + troco, regressa ao idle

DISPENSAÇÃO – MATRIZ LED + SENSORES

normal fluxo esperado **erro** entrada inválida ou falha **limite** caso extremo **auto** comportamento automático

normal **Dispensação bem-sucedida**
IR deteta produto no tabuleiro ou sensor de posição ativa → "Thank you!"

normal **IR já bloqueado (produto em tabuleiro)**
Usa apenas sensor de posição para confirmar rotação completa da espiral

limite **Ciclo 2 de tentativa**
Motor tenta 2ª rotação se ciclo 1 falhar; se IR ativar no ciclo 2 → sucesso

erro **Timeout motor**
Sem detecção após 2 ciclos → "Dispense error! Contact support."

normal **Troco correto após venda**
balance -= price antes de Payment_Reset(), troco enviado via UART3

REFRIGERAÇÃO - DHT11 + TIM6 + LED AZUL

normal fluxo esperado

erro entrada inválida ou falha

limite caso extremo

auto comportamento automático

auto

Leitura periódica DHT11

TIM6 dispara a cada 2s → Refrig_Update() lê temperatura e humidade

auto

Arrefecimento liga ($>7^{\circ}\text{C}$)

Motor de cooling ativa, alerta BT enviado na transição

auto

Arrefecimento desliga ($<3^{\circ}\text{C}$)

Motor desativa, info BT enviado na transição (histerese $3-7^{\circ}\text{C}$)

erro

Falha sensor DHT11

TIMEOUT / ERROR / CHECKSUM → alerta BT, ciclo ignorado sem crash

limite

Temperatura crítica

Acima do limite crítico → cooling forçado ON + alerta crítico BT

COMANDOS BLUETOOTH – UART1

normal fluxo esperado

erro entrada inválida ou falha

limite caso extremo

auto comportamento automático

normal

STOCK:STATUS

Devolve tabela completa 4x4 com quantidades, preços e validades

normal

STOCK:LxCx

Estado de um produto individual (ex: STOCK:L1C2)

normal

STOCK:LxCx:QTY:n

Atualiza quantidade (ex: STOCK:L1C2:QTY:5), confirma via BT

normal

STOCK:LxCx:VAL:DD/MM/YY

Atualiza validade (ex: STOCK:L1C2:VAL:30/12/27), confirma via BT

normal

TEMP

Devolve temperatura e humidade atuais com timestamp RTC

normal

VENDAS

Devolve número de vendas e receita total acumulada

normal

AJUDA

Lista todos os comandos disponíveis

erro

Comando inválido

Ignorado silenciosamente, nenhuma flag ativa

erro

Índice fora do intervalo

STOCK:L0C5 ou similar → rejeitado por validação de range

ILUMINAÇÃO – RTC + LED AMARELO

normal

fluxo esperado

erro

entrada inválida ou falha

limite

caso extremo

auto

comportamento automático

auto

Ligar ao arranque

Se hora atual entre LIGHTS_ON e LIGHTS_OFF → liga imediatamente

auto

Alarme A (07:00)

RTC dispara → luzes ligam automaticamente

auto

Alarme B (20:00)

RTC dispara → luzes apagam automaticamente

DISPLAY - LCD

normal

fluxo esperado

erro

entrada inválida ou falha

limite

caso extremo

auto

comportamento automático

auto

Rotação idle

Alterna "Welcome!" → hora+temperatura a cada 2s quando sem saldo e sem preço

auto

Rotação suspensa com saldo

Quando balance>0 ou price>0, rotação para e ecrã de saldo/preço mantém-se

normal

Ecrã de seleção (1º dígito)

"Row: X" + saldo na linha 2

CONCLUSÃO

O projeto permitiu desenvolver um sistema de máquina de venda automática com base no microcontrolador STM32, abrangendo funcionalidades como pagamento por identificação de moedas, dispensação por espiral, gestão de stock com controlo de validades, refrigeração com histerese e monitorização remota via Bluetooth.

Uma das principais dificuldades foi a necessidade de ter especial cuidado com as configurações dos periféricos: timers, GPIO, UARTs e RTC.

Como trabalho futuro, sugeríamos a implementação da autenticação nos comandos Bluetooth e o desenvolvimento de um ficheiro de informações em memória não volátil.

Resumindo, o projeto consolidou competências como a programação de periféricos, a gestão de interrupções, a comunicação série e a estruturação de software modular.

